

How the spaceplane lander works

From low Kerbin orbit to a stop on the KSC runway: one deorbit burn and then 1,500 km of unpowered flight, from a kRPC script

The vehicle starts in orbit with an engine, a little propellant and no further thrust after one burn. The autopilot chooses when and how hard to burn, flies a hypersonic entry with angle of attack and bank, spirals off its leftover energy in a circle over the field, glides down final, flares and brakes to a stop. The runway is 2.4 km long and 70 m wide. On the target vehicle, a Mk3 shuttle, it lands intact on the runway in about three flights of four from a save taken over the field. From orbit it is still the open problem.

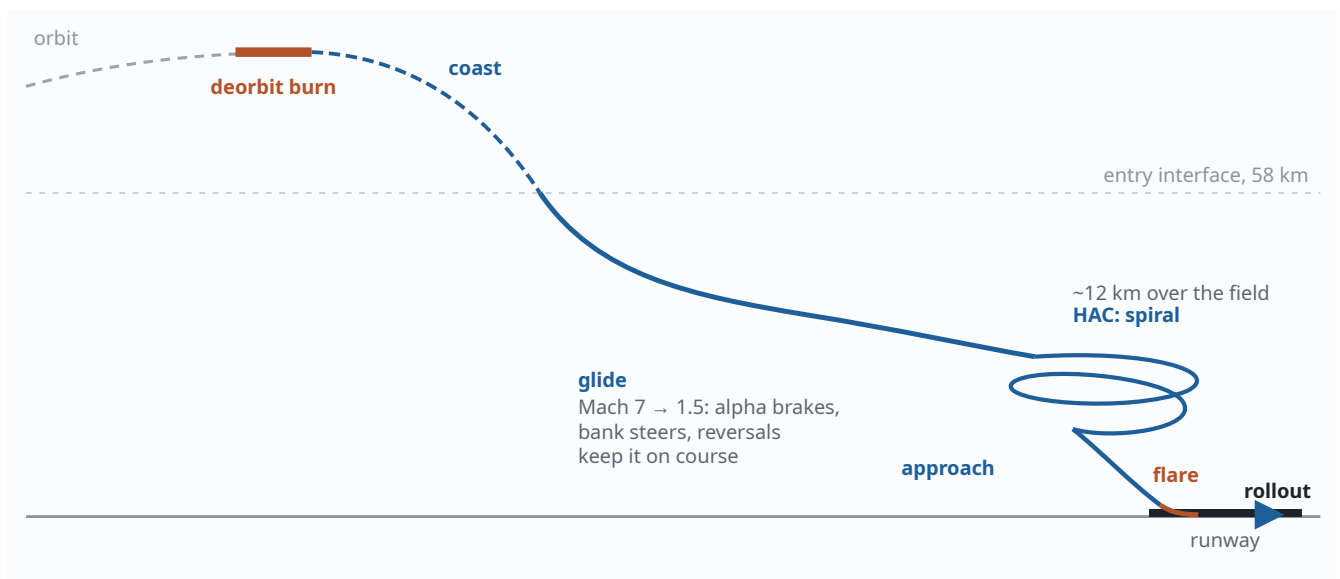


Figure 1. The flight profile, heavily not to scale: the glide covers well over a thousand kilometres, the circle over the field a few tens. Only the deorbit burn uses the engine.

The architecture

The spaceplane shares the booster lander's skeleton. One phase machine (`autopilot.py`) is the only code that talks to the game. **Every tick it re-propagates the vehicle to the ground** with a local RK4 integrator (`trajectory.py`) that models gravity, lift, drag and the planet's rotation, and every phase decision comes from that fresh prediction rather than from a stored plan. The control laws are pure functions in `guidance.py`. `environment.py` holds the air, the runway and the airframe's aerodynamic table, so the integrator never makes a remote call.

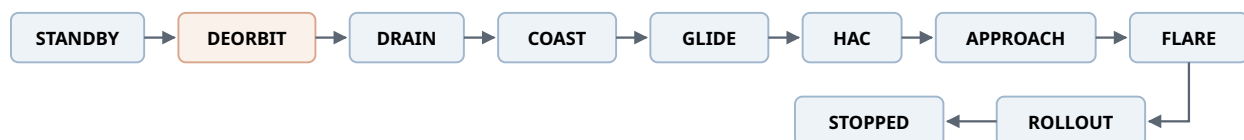


Figure 2. The phase machine. With the default `DRAIN_BEFORE_BURN`, DRAIN runs just before the burn: surplus propellant is dumped in orbit, leaving a reserve the burn spends. Phases hand over at altitudes and geometry, never at a miss distance.

Three ideas recur through the whole design.

- **Measure the airframe; don't type it in.** The lift and drag table is swept in STANDBY with kRPC's `simulate_aerodynamic_force_at`, 182 probe calls in under a second, so the first prediction already uses this vehicle's coefficients. The probe takes the airflow as an argument, so each cell is measured at exactly the angle of attack it tabulates. The vehicle's own axes are checked against `Vessel.direction` before anything is trusted. The highest angle of attack the airframe can actually *hold* is learned in flight (see the glide).
- **Surplus energy is recoverable; a deficit is not.** A glider can always throw energy away by banking, pitching up or S-turning, but it can never make more. So every aim in the flight sits deliberately on the long side, and the gates are placed above the best glide from them, not below.
- **The predictor flies the law the vehicle flies.** The speed cap, the bank-reversal behaviour and the alpha ceiling are applied inside the propagator as well as in the control loop. A prediction of a law nobody flies is a prediction of a trajectory nobody flies.

Deorbit: choosing the one burn

In orbit the autopilot asks each tick whether the runway is within deorbit range, time-warping while it isn't. Inside the range it searches candidate burn sizes, propagating a full entry for each, and commits when one works. What “works” means is the important part.

The glide that follows has a finite **window** of reachable arrivals. Its shortest is the steepest, highest-drag entry the glide solver is allowed to fly, and its longest is the most lifting one. `guidance.deorbit_window` propagates those two corners, and the burn is chosen to put the target **in the middle of the window**. Because the corners are exactly the commands the glide may issue, the window is the glide's real authority rather than a guess about the airframe. This replaced a fitted “aim long by X% of the arc” constant that had to be re-fitted five times in one session, each time forced by an unrelated change elsewhere.

The burn itself is closed-loop: it points retrograde, and instead of asking “how much more?” it asks each tick whether the trajectory *already* does the job (one propagation, not sixteen). The engine's thrust limiter is turned down so small burns last at least three seconds and can be cut off accurately. The flip to burn attitude uses RCS, because it is on the critical path; flying it on reaction wheels alone cost 50 km on every arrival.

WHY THE BURN MATTERS SO MUCH

One metre per second at the entry interface is worth roughly 11 km on the ground. Fifteen flights from three saves taken *at* the interface repeated to within 2.3 km, against 14.8 km for whole flights from orbit: as variance, **97.6% of the along-track scatter is made before the entry starts**. The single largest fix to it was not in guidance at all. The propellant drain valve had been firing *after* the burn's closed loop stopped looking, giving a small, unmeasured push. Draining before the burn closed the gap between entry states.

Coast

After the burn the vehicle falls to the 58 km entry interface. It is already in the entry attitude (22° angle of attack), because an attitude set up in vacuum costs nothing to hold, while one set up in thickening air has to be fought for. RCS is permitted, and time warp is used where nothing needs control.

Glide: angle of attack brakes, bank steers

From the interface down to about Mach 1.5, the glide re-solves its two controls every tick to arrive over the field. The work is split between them because the measurements force it:

CONTROL	WHAT IT DOES
angle of attack	Downrange and energy. Drag at 30° is ten times drag at 0°. Solved against the predicted along-track miss.
bank magnitude	Rolling lift off the vertical lets the vehicle sink into thicker air, which is what really slows it. Takes whatever the angle of attack leaves.
bank sign	The <i>only</i> cross-track authority in the whole flight. The sign flips (a bank reversal) whenever the heading error to the runway leaves a deadband, and the deadband narrows as the range shrinks.

The obvious design, solving both controls together through a 2×2 Jacobian as the booster's wing does, failed here. The sensitivity of range to angle of attack **changes sign during the entry**: at 80 km more alpha lengthens the flight, and by 45 km it shortens it. Range is also non-monotone in both controls at the shallow end: maximum L/D gives the *shortest* flight, and a little bank lengthens it. The solver brackets to stay on the monotone side of each.

The propagator models a reversing entry as its average: vertical lift reduced by $\cos(\text{bank})$, with no net sideways lift. Modelling one fixed lean instead predicted a huge curving arc whose cross-track sign flipped every cycle, which commanded a vehicle rolling through 40° twice a second.

The ceiling the airframe will actually hold

In dense air the vehicle doesn't hold what it is told: asked for 20° at Mach 4 it delivers 15–18°, and the shortfall grows with dynamic pressure. That is a **saturation**, *achieved* = $\min(\text{command}, \text{holdable}(q))$, not a gain, and the difference matters. With a proportional model the solver simply asks for more, the factor cancels, and the prediction believes a lift that never arrives. kRPC won't report a pitching moment, so the ceiling can't be probed. `trajectory.Holdable` learns it instead from every tick's command-versus-achieved pair, binned by dynamic pressure. A bin with no evidence returns `None`, and queries beyond anything seen extrapolate *downward* only. Predicting too little lift makes the burn larger, and surplus is the recoverable side.

Arriving at the right speed

Hold the entry angle of attack to the end and the vehicle slows to that angle's equilibrium speed, which is below stall. A speed cap therefore takes over near the end and uses angle of attack to hold the approach speed. It is two-sided below a set Mach number, because a one-sided cap disengages exactly when the vehicle dives, and a dive turns surplus height into speed that nothing after the gate can spend. One flight arrived at the approach doing 219 m/s, 74° off the runway heading.

HAC: stop asking the entry for an exact energy

This is the change that took the landing from tens of kilometres to hundreds of metres. It works by changing what the entry is asked to do, not by doing it more accurately. A straight-in arrival has to be at one point, one height and one energy after 1,500 km of unpowered flight, and the burn alone scatters that by ~15 km. The Space Shuttle didn't try. It arrived *over* the field with height in hand and spent the surplus turning round a **heading alignment cone** (HAC), a circle tangent to the extended

centreline. Rolling out of the circle is lining up, so alignment and energy management become the same manoeuvre.

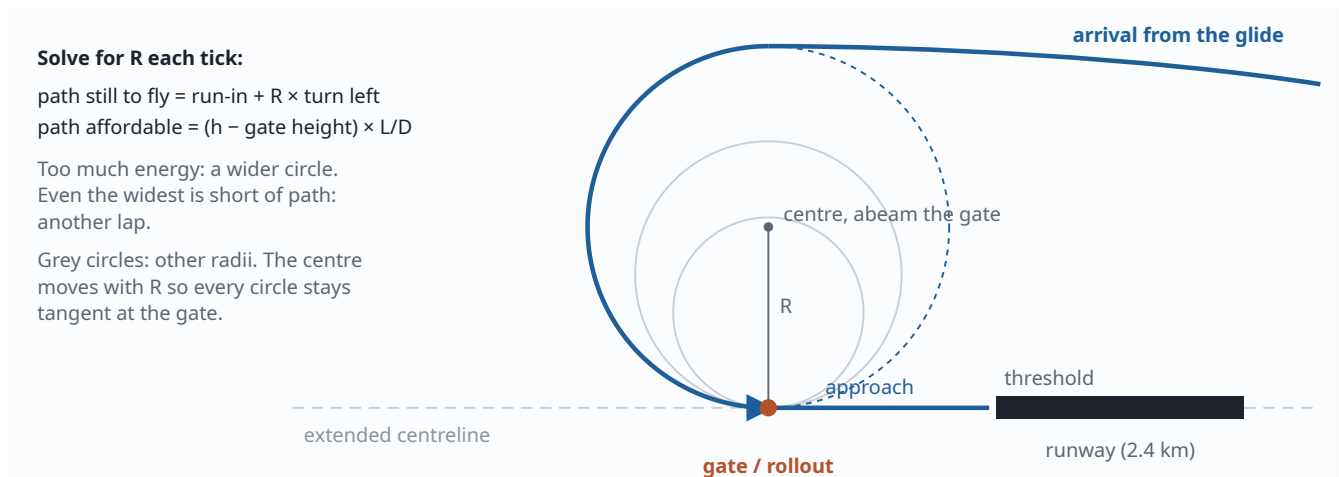


Figure 3. Plan view of the heading alignment cone. An energy error becomes a radius, and a radius is something the vehicle can just fly. The circle is entered only below Mach 1.5: turn radius goes as v^2 , about 9 km at Mach 0.9 and 250 km at Mach 4.7.

Making this work cost several flights, and each lesson is now a rule in the code:

- **The centre moves with the radius.** With the centre pinned 8 km abeam, every other radius missed the gate sideways by $(8 \text{ km} - R)$. Three flights landed 0.2–1.6 km off the centreline of a 70 m runway. With the centre moved, the next four landed within 200 m.
- **The tangent point is reached travelling the right way round.** Both tangent points satisfy the arithmetic. With the wrong sign, a vehicle already lined up 30 km out was costed a 149° turn and abandoned the cone.
- **Lined up is not ready.** Reaching the rollout with a lap still owed, or with more height than the approach can lose, is not an exit. Two early flights handed the approach a vehicle 12 km up and floated 10 km past the field.
- **The entry's target moves up.** The glide now aims at a point ~12 km over the gate instead of at the gate itself. The vehicle crosses the field at Mach 2.9 at 19 km, so the cone has to be delivered where it can be flown, and the raised target gives back ~19 km of required range the entry was carrying as surplus anyway.

Approach: hold speed, not path

A glider can't choose its speed and its flight path independently: pick an angle of attack and the polar fixes both. The one to choose is **speed**, because arriving slow can't be recovered. The first approach law used the throttle-aircraft logic of commanding alpha from sink-rate error, which is backwards here. Arriving high, it unloaded the wing and traded the height for speed, and offline reached the threshold at 102 m/s and 36 m/s of sink.

The current law commands the *descent angle* that holds the target speed, from $dv/dt = g \sin\theta - D/m$. That lets it make speed by steepening as well as lose it. Surplus height is spent with S-turns that command a track angle, $\arccos(\text{straight path} / \text{path the height can pay for})$, rather than a bank. The target speed is deliberately **fast**, $2.25\times$ stall. The load factor available to arrest the sink in the flare grows as the *square* of speed ($n_{\text{max}} = (V/V_s)^2$), while the sink to be arrested grows only linearly, so a faster approach makes the flare strictly easier. Horizontal speed is the cheap axis, and the runway is where it is meant to go.

Flare and rollout: no law may step

Four separate laws in the last few hundred metres were commanding *steps*, and each one destroyed the vehicle on a runway it had already reached. The common rule: **ramp every command, and near the ground reference it to the runway, not the airflow.**

- **Flare (from 50 m).** The flare tracks a sink rate of $\sqrt{v_{td}^2 + 2ah}$, easing smoothly to the touchdown rate. Its arrest demand divides by height, so a floor stops it from demanding maximum lift over the last two ticks, and a touchdown attitude caps pitch short of a tail strike. Below 30 m it aligns the nose with the runway, not with the velocity. Pointing at the velocity landed the vehicle crabbed: 23° of sideslip at the first rollout tick, 51° two seconds later, gear torn off.
- **Centreline capture.** The approach asks for the lateral closure that brings cross-track to zero *when the wheels arrive*, not just “inside the offset left”. The flare has only about 1.5 s of lateral authority, so it inherits whatever drift it is handed.
- **Rollout.** Ramp the nose down from the attitude it touched down in, instead of commanding zero alpha at 90 m/s (a nose-over) or nothing at all (a tail scrape). Full main-gear braking from touchdown, nosewheel steering onto the centreline, ground spoilers. STOPPED hands the vehicle back.

Where it stands (2026-10-05)

SEGMENT	STATE
deorbit + entry	Works on both airframes. The shuttle's arrival error went from +111 km to +3 km after attitude and burn fixes, including the engine thrust limiter, which had never been commanded.
landing, from saves over the field	Shuttle on the runway intact in 23 of 30 flights across five save states (36 with a sixth, hard one included: 23/36). Touchdown cross-track is now a median of 13–20 m.
landing, from orbit	The current blocker: about 1 orbital flight in 8 lands on the runway. The cone reaches the rollout 300–3,500 m too high, and its exit prices energy against a gate 6 km out while the vehicle is already beside the field.
touchdown survival	The shuttle's wings were breaking off on contact. The cause was the <code>rigidAttachment</code> flag on the two delta-wing parts: with it off, 22 of 24 touchdowns kept both wings, against ~8 of 24.

How it is measured

Changes are flown on a farm of six headless KSP instances, with the autopilot unmodified. A batch is n flights of one configuration, not one flight of n configurations: one unchanged configuration scatters 10–15 km along-track from orbit, and an 8-flight batch resolves a factor of two and nothing finer. Arms are interleaved round by round so drift in the farm cancels. Every log records the settings that differ from the defaults *and* a fingerprint of the defaults, and ends with a `loop rate` line showing whether each phase's control loop actually ran at the interval it asked for. Landing work is flown from saves taken over the field, since a landing can't be measured through 15 km of entry scatter. A headless simulator (`kspSim`) screens ideas about ten times faster, and the farm confirms them.